[Logo del instituto]

INSTITUTO TÉCNICO DE EDUCACIÓN COMERCIAL AMERICANO
Programación de Dispositivos Móviles II

TiendaCerca

Aplicación móvil para la búsqueda de tiendas cercanas y sus ofertas

Documentación del proyecto — entrega final

Presentado por:
Roberto

Docente: Ing. Alexander Pabon Pabon
Gestión 2026
20 de octubre de 2026

Índice

1. Descripción del Proyecto

2. Objetivos

3. Público Objetivo

4. Funcionalidades

5. Herramientas Utilizadas

6. Diagramas

7. Manual de Usuario

8. Registro y Gestión de Errores

9. Conclusiones y Trabajo Futuro

10. Bibliografía

1. Descripción del Proyecto

TiendaCerca es una aplicación móvil desarrollada en Flutter que permite a cualquier persona buscar tiendas y comercios cercanos (ferreterías, abarrotes, ropa, farmacias, electrónica, restaurantes, entre otros) usando su ubicación real, y ver las ofertas vigentes que cada tienda publica. Resuelve el problema de que, en ciudades como La Paz, un cliente no siempre sabe qué tiendas cercanas existen ni qué ofertas tienen en un momento dado, mientras que los pequeños comerciantes rara vez cuentan con un canal digital propio para anunciar sus promociones.

---

2. Objetivos

2.1 Objetivo general

Desarrollar una aplicación móvil en Flutter que conecte a clientes con tiendas y comercios cercanos mediante geolocalización real, permitiendo que cada tienda publique y administre sus propias ofertas.

2.2 Objetivos específicos

- Implementar la búsqueda y el filtrado de tiendas cercanas por categoría, usando la ubicación real del usuario (GPS).
- Permitir que cada Dueño de tienda registre su propio negocio (nombre, categoría, ubicación en el mapa y foto) y publique, edite, pause o elimine sus ofertas.
- Incorporar autenticación real de usuarios (registro e inicio de sesión con contraseña) y persistencia local de todos los datos con SQLite.

---

3. Público Objetivo

La app está pensada para dos tipos de usuario: clientes que buscan comprar en comercios locales cercanos sin conocer necesariamente la zona de antemano, y dueños de pequeños negocios

(ferreterías, abarrotes, tiendas de ropa, farmacias, electrónica, restaurantes, etc.) que no cuentan con un canal digital propio para anunciar sus ofertas y promociones.

4. Funcionalidades

- Búsqueda de tiendas cercanas en un mapa real (OpenStreetMap), filtrable por categoría y por texto, ordenada por distancia real.
- Registro de una tienda propia con ubicación elegida en el mapa y una foto, al crear una cuenta como Dueño de tienda.
- Publicación, edición, pausa y eliminación de ofertas con foto, precio y descripción.
- Favoritos, para guardar ofertas de interés y encontrarlas de nuevo fácilmente.
- Panel de Administrador para ver todas las tiendas de la plataforma y moderar (pausar o eliminar) cualquier oferta publicada.

---

5. Herramientas Utilizadas

Categoría	Herramienta
Lenguaje	Dart
Framework	Flutter
IDE	Visual Studio Code
Manejo de estado	provider (ChangeNotifier)
Base de datos local	sqflite (SQLite)
Geolocalización	geolocator
Mapa	flutter_map + latlong2 (OpenStreetMap)
Fotos	image_picker (cámara y galería)
Seguridad de contraseñas	crypto (hash SHA-256)
Llamar / abrir mapas	url_launcher

6. Diagramas

6.1 Flujo de pantallas

Tras iniciar sesión o crear una cuenta, la app arma las pestañas visibles según el rol de la cuenta: Cliente, Dueño de tienda o Administrador.

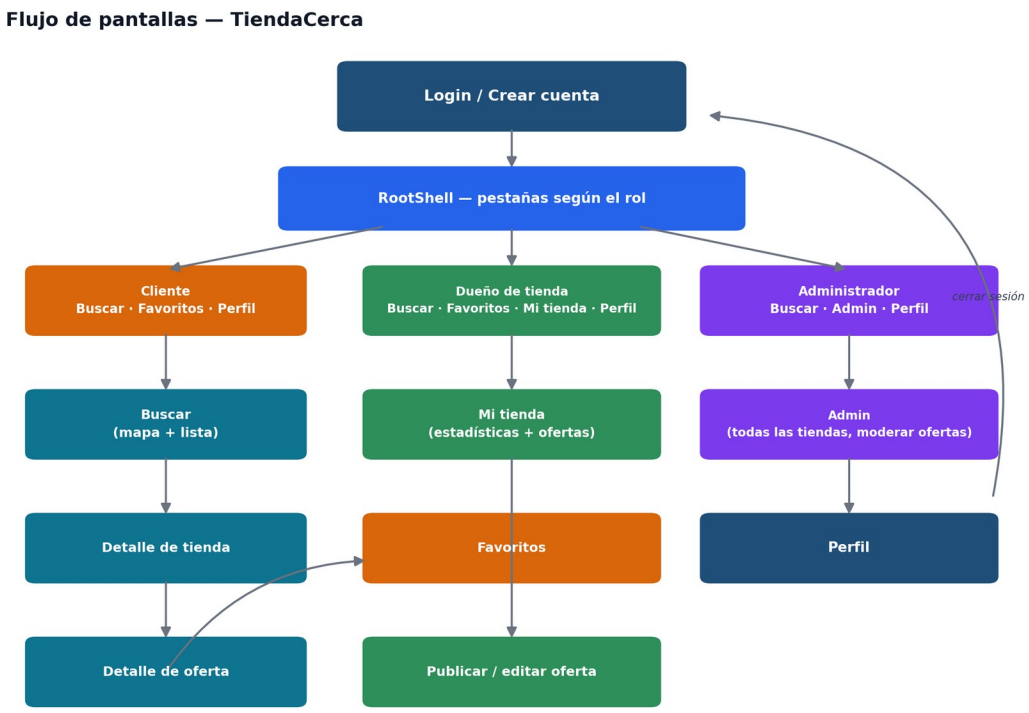


Figura 1. Flujo de pantallas de TiendaCerca.

6.2 Diagrama de base de datos

Toda la información que el usuario genera de verdad (cuenta, sesión, tienda propia, ofertas y favoritos) se guarda en una base de datos SQLite local con 5 tablas.

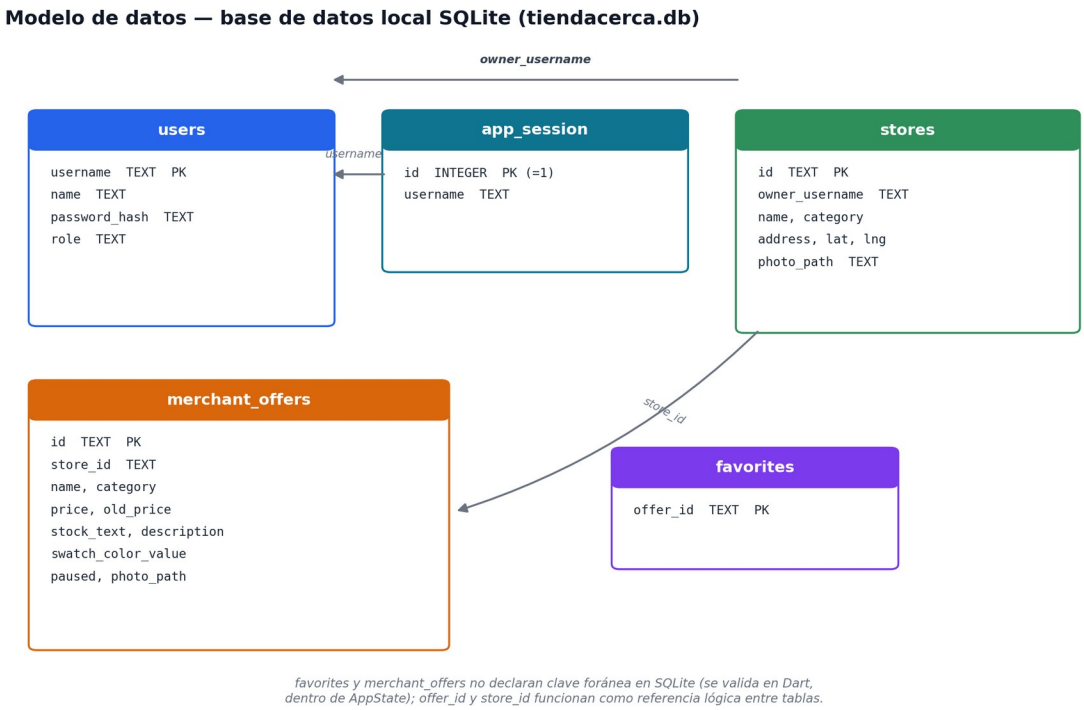


Figura 2. Modelo de datos de la base SQLite local (tiendacerca.db).

6.3 Cronograma del proyecto

El proyecto se desarrolló en 8 semanas, siguiendo la bitácora de seguimiento de la materia, desde la idea inicial hasta el cierre y esta documentación final.

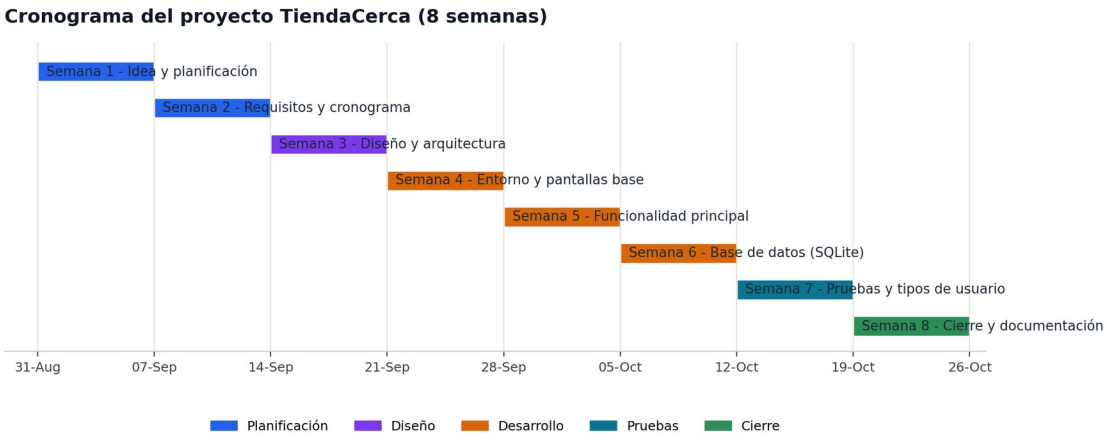


Figura 3. Cronograma de 8 semanas del proyecto TiendaCerca (31 de agosto – 25 de octubre de 2026).

7. Manual de Usuario

Pasos básicos para usar TiendaCerca:

1. Abre la app y crea una cuenta (o inicia sesión si ya tienes una), eligiendo el rol Cliente o Dueño de tienda.
2. En la pestaña Buscar, explora el mapa o la lista de tiendas cercanas y filtra por categoría.
3. Toca una tienda para ver su detalle: dirección, horario, teléfono y sus ofertas vigentes.
4. Toca el corazón de una oferta para guardarla en Favoritos.
5. Si tu cuenta es Dueño de tienda, abre la pestaña Mi tienda y toca "Publicar oferta" para agregar una oferta nueva con foto, precio y descripción.

[Insertar aquí 2–3 capturas de pantalla reales: Buscar, Detalle de tienda y Mi tienda / Publicar oferta.]

8. Registro y Gestión de Errores

El registro y la gestión de errores son una parte importante del desarrollo de TiendaCerca, ya que permiten identificar fallos reales, documentar sus causas y dejar constancia de cómo se corrigieron. A diferencia de un listado de defectos simulados, esta sección recopila los errores que efectivamente se encontraron durante las 8 semanas de desarrollo del proyecto.

8.1 Registro de defectos e incidencias

ID	Fecha	Módulo	Error identificado	Pasos para reproducir	Prioridad	Estado
BUG-01	23/09/2026	Configuración / Build	La app no compilaba ("algo salió mal") por un choque de nombres y permisos faltantes.	1. Ejecutar flutter run. 2. Ver el error de compilación. 3. Revisar el analizador: conflicto entre una clase propia y ColorSwatch de Flutter. 4. Revisar AndroidManifest.xml: faltan los permisos de ubicación.	Alta	Cerrado
BUG-02	07/10/2026	Detalle de tienda	Las ofertas publicadas por un Dueño de tienda no aparecían al verlas como Cliente.	1. Iniciar sesión como Dueño de tienda. 2. Publicar una oferta nueva. 3. Cerrar sesión e iniciar como Cliente. 4. Abrir el detalle de esa tienda. 5. Ver que la oferta publicada no aparece en la lista.	Alta	Cerrado
BUG-03	08/10/2026	Publicar oferta	El botón de agregar foto a una oferta no hacía nada real (solo cambiaba de color).	1. Abrir Publicar oferta. 2. Tocar el recuadro de foto. 3. Ver que solo cambia de color, sin abrir la cámara ni la galería.	Media	Cerrado
BUG-04	09/10/2026	Buscar / Ubicación	Un fallo de geolocalización podía tumbar la app.	1. Desactivar el permiso de ubicación del dispositivo. 2. Abrir la pestaña Buscar. 3. Ver que la app lanza una excepción no controlada en vez de usar un punto de respaldo.	Alta	Cerrado
BUG-05	14/10/2026	Mi tienda	No se podía editar ni eliminar una oferta publicada, solo pausarla.	1. Iniciar sesión como Dueño de tienda. 2. Ir a Mi tienda. 3. Intentar modificar o quitar una oferta ya publicada. 4. Ver que solo existe la opción de pausar/reactivar.	Media	Cerrado
BUG-06	15/10/2026	Interfaz general	El analizador mostraba advertencias por el uso de APIs obsoletas de Flutter.	1. Ejecutar flutter analyze. 2. Ver advertencias sobre withOpacity y el parámetro value: de DropdownButtonFormField.	Baja	Cerrado
BUG-07	19/10/2026	Autenticación / Login	El inicio de sesión con un código especial no era un login real (sin usuario ni contraseña).	1. Abrir la app. 2. Elegir un rol y escribir "admin123" en el campo de nombre. 3. Ver que se entra como Administrador sin validar una cuenta real.	Alta	Cerrado
BUG-08	20/10/2026	Detalle de tienda	StoreDetailScreen podía reventar con un StateError si el id de la tienda no existía.	1. Navegar al detalle de una tienda usando un storeId que ya no existe. 2. Ver que la app lanza un StateError en vez de mostrar un mensaje.	Alta	Cerrado
BUG-09	20/10/2026	Configuración / Android	El nombre de la app en el launcher del teléfono seguía diciendo "ferrecerca_app".	1. Instalar la app en un celular. 2. Revisar el nombre debajo del ícono en el launcher. 3. Ver que dice "ferrecerca_app" en vez de "TiendaCerca".	Baja	Cerrado
BUG-10	20/10/2026	Mapa / Buscar	El userAgentPackageName del mapa no coincidía con el	1. Abrir la pestaña Buscar. 2. Revisar la configuración del TileLayer del mapa.	Baja	Cerrado

			applicationId real de la app.	3. Comparar el userAgentPackageName con el applicationId real del proyecto.		

Observación: la fecha corresponde a la semana aproximada en que se detectó cada error durante el desarrollo real del proyecto. El estado "Cerrado" indica que la corrección fue aplicada y verificada con una prueba posterior.

8.2 Procedimiento para gestionar un error

1. Encontrar el error: usar las funciones principales de la app para detectar un comportamiento que no corresponde a lo esperado.
2. Reproducir el error: repetir los mismos pasos y datos para confirmar que ocurre de forma consistente.
3. Documentar la incidencia: registrar id, fecha, módulo, descripción, pasos, prioridad y estado.
4. Identificar la causa: revisar la lógica de AppState, las consultas a DatabaseHelper, las validaciones o los permisos del dispositivo, según corresponda.
5. Corregir el error: modificar el código responsable sin afectar otra funcionalidad que ya estuviera funcionando.
6. Ejecutar nuevamente la prueba: repetir los pasos originales y revisar funcionalidades cercanas para descartar un problema nuevo.
7. Registrar el resultado: actualizar el estado a Cerrado si la prueba confirma la solución, o dejarlo abierto si el problema persiste.

8.3 Ciclo de vida de un error

Nuevo → Asignado → En desarrollo → Resuelto → En pruebas → Cerrado

- Nuevo: el defecto se acaba de detectar y registrar en la tabla.
- Asignado: se define quién se encarga de corregirlo (en un proyecto individual, el mismo desarrollador).
- En desarrollo: se analiza la causa y se modifica el código responsable.
- Resuelto: la corrección ya está lista para volver a probarse.
- En pruebas: se repiten los pasos originales para confirmar que el error desapareció.
- Cerrado: las pruebas confirman que el defecto quedó solucionado y no reaparece.

Si un error corregido vuelve a presentarse durante una prueba posterior, la incidencia pasa al estado Reabierto y regresa al análisis y la corrección.

8.4 Ejemplo de seguimiento de una incidencia

Identificador: BUG-02

Módulo: Detalle de tienda.

Descripción: las ofertas que un Dueño de tienda publicaba no se veían reflejadas al revisar esa tienda desde una cuenta Cliente.

Causa posible: las pantallas de comprador todavía leían el catálogo fijo de ejemplo (MockData) en vez de los datos reales guardados en SQLite.

Corrección propuesta: se agregaron los métodos AppState.offersForStore()/findOffer() como única fuente de verdad para las ofertas de una tienda.

Prueba posterior: publicar una oferta nueva como Dueño de tienda, cerrar sesión, entrar como Cliente y abrir el detalle de esa misma tienda. El resultado esperado es que la oferta publicada aparezca de inmediato en la lista.

Estado final: Cerrado — confirmado mediante la prueba posterior descrita arriba.

9. Conclusiones y Trabajo Futuro

9.1 Conclusiones

Desarrollar TiendaCerca permitió aplicar de forma práctica el manejo de estado con Provider, la persistencia local con SQLite, la geolocalización real, los permisos de dispositivo (ubicación y cámara) y la autenticación real de usuarios con distintos roles. La arquitectura por capas del proyecto permitió agregar funcionalidad nueva — como el login real o que cada Dueño de tienda tenga su propia tienda — sin tener que reescribir las pantallas ya construidas.

9.2 Trabajo futuro

- Conectar un backend remoto real (Firebase, Supabase o una API propia) para el catálogo general de tiendas.
- Agregar notificaciones push cuando una tienda que el usuario sigue publica una oferta nueva.
- Permitir que un Dueño de tienda edite su tienda después de registrarla (hoy el nombre, la ubicación y la foto se fijan al crear la cuenta).
- Ampliar las pruebas automatizadas para cubrir también los flujos de editar y eliminar una oferta de punta a punta.

---

10. Bibliografía

Flutter. (2026). Flutter documentation. Google. https://docs.flutter.dev

provider (paquete Dart). (2026). pub.dev. https://pub.dev/packages/provider

sqflite (paquete Dart). (2026). pub.dev. https://pub.dev/packages/sqflite

geolocator (paquete Dart). (2026). pub.dev. https://pub.dev/packages/geolocator

flutter_map (paquete Dart). (2026). pub.dev. https://pub.dev/packages/flutter_map

OpenStreetMap contributors. (2026). OpenStreetMap. https://www.openstreetmap.org

image_picker (paquete Dart). (2026). pub.dev. https://pub.dev/packages/image_picker

crypto (paquete Dart). (2026). pub.dev. https://pub.dev/packages/crypto

Anthropic. (2026). Claude (modelo de lenguaje de IA). Usado como asistente para apoyar la redacción de la documentación, la generación de los diagramas y la revisión de código durante el desarrollo del proyecto. https://claude.ai